ds_read_tr

Nicola Zaghen – October 2025

AMD
together we advance_

Lowering local_load

- Problem
  - Lots of small ds_read when we load from LDS in MFMA layout when using local_load
  - Happens usually when ordering in LDS is column major but we want to use data in row major (or vice versa)
  - This is because we store the data the same way as the blocked layout
  - Could store data in row major in LDS but this would cause a lot of small ds_write ➔ not a good solution

- Solution
  - Can we read the LDS allocation with the opposite ordering?
  - Yes! The instruction ds_read_tr allows exactly that
  - Supports different data types: ds_read_tr16_b64, ds_read_tr8_b64, ds_read_tr4_b64, ds_read_tr6_b96
  - Each of these instructions loads a total of 64 bits of data for each instance
  - An MFMA tile requires 128 bits of data, so we need to issue this instruction twice get enough data for a full tile

NOTE: Contrary to the name, ds_read_tr does not transpose data. It shuffles it among a group of lanes

Thread address map MFMA layout vs ds_read_tr16_b64

M (A Matrix) N (B Matrix)	K 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	Thread 0								Thread 16							
1	Thread 1								Thread 17							
2	Thread 2								Thread 18							
3	Thread 3								Thread 19							
4	Thread 4								Thread 20							
5	Thread 5								Thread 21							
6	Thread 6								Thread 22							
7	Thread 7								Thread 23							
8	Thread 8								Thread 24							
9	Thread 9								Thread 25							
10	Thread 10								Thread 26							
11	Thread 11								Thread 27							
12	Thread 12								Thread 28							
13	Thread 13								Thread 29							
14	Thread 14								Thread 30							
15	Thread 15								Thread 31							

M (A Matrix) N (B Matrix)	K 16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	Thread 32								Thread 48							
1	Thread 33								Thread 49							
2	Thread 34								Thread 50							
3	Thread 35								Thread 51							
4	Thread 36								Thread 52							
5	Thread 37								Thread 53							
6	Thread 38								Thread 54							
7	Thread 39								Thread 55							
8	Thread 40								Thread 56							
9	Thread 41								Thread 57							
10	Thread 42								Thread 58							
11	Thread 43								Thread 59							
12	Thread 44								Thread 60							
13	Thread 45								Thread 61							
14	Thread 46								Thread 62							
15	Thread 47								Thread 63							

Figure 1 Matrix A Row Major A[16][32] or B Column Major B[32][16] (16b)

M (A Matrix) N (B Matrix)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	Thread 0	Thread 4	Thread 8	Thread 12	Thread 0	Thread 4	Thread 8	Thread 12	Thread 16	Thread 20	Thread 24	Thread 28	Thread 16	Thread 20	Thread 24	Thread 28
1	Thread 1	Thread 5	Thread 9	Thread 13	Thread 1	Thread 5	Thread 9	Thread 13	Thread 17	Thread 21	Thread 25	Thread 29	Thread 17	Thread 21	Thread 25	Thread 29
2	Thread 2	Thread 6	Thread 10	Thread 14	Thread 2	Thread 6	Thread 10	Thread 14	Thread 18	Thread 22	Thread 26	Thread 30	Thread 18	Thread 22	Thread 26	Thread 30
3	Thread 3	Thread 7	Thread 11	Thread 15	Thread 3	Thread 7	Thread 11	Thread 15	Thread 19	Thread 23	Thread 27	Thread 31	Thread 19	Thread 23	Thread 27	Thread 31
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																

M (A Matrix) N (B Matrix)	K 16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	Thread 32	Thread 36	Thread 40	Thread 44	Thread 32	Thread 36	Thread 40	Thread 44	Thread 48	Thread 52	Thread 56	Thread 60	Thread 48	Thread 52	Thread 56	Thread 60
1	Thread 33	Thread 37	Thread 41	Thread 45	Thread 33	Thread 37	Thread 41	Thread 45	Thread 49	Thread 53	Thread 57	Thread 61	Thread 49	Thread 53	Thread 57	Thread 61
2	Thread 34	Thread 38	Thread 42	Thread 46	Thread 34	Thread 38	Thread 42	Thread 46	Thread 50	Thread 54	Thread 58	Thread 62	Thread 50	Thread 54	Thread 58	Thread 62
3	Thread 35	Thread 39	Thread 43	Thread 47	Thread 35	Thread 39	Thread 43	Thread 47	Thread 51	Thread 55	Thread 59	Thread 63	Thread 51	Thread 55	Thread 59	Thread 63
4																
5																
6																
7																
8																
9																
10																
11																
12																
13																
14																
15																

Figure 3 Matrix A Column Major [16][32] or Matrix B Row Major [32][16] (16b)

Linear Layout for MFMA

		K															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
M (A Matrix) N (B Matrix)	0	Thread 0								Thread 16							
	1	Thread 1								Thread 17							
	2	Thread 2								Thread 18							
	3	Thread 3								Thread 19							
	4	Thread 4								Thread 20							
	5	Thread 5								Thread 21							
	6	Thread 6								Thread 22							
	7	Thread 7								Thread 23							
	8	Thread 8								Thread 24							
	9	Thread 9								Thread 25							
	10	Thread 10								Thread 26							
	11	Thread 11								Thread 27							
	12	Thread 12								Thread 28							
	13	Thread 13								Thread 29							
	14	Thread 14								Thread 30							
	15	Thread 15								Thread 31							
		K															
		16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
M (A Matrix) N (B Matrix)	0	Thread 32								Thread 48							
	1	Thread 33								Thread 49							
	2	Thread 34								Thread 50							
	3	Thread 35								Thread 51							
	4	Thread 36								Thread 52							
	5	Thread 37								Thread 53							
	6	Thread 38								Thread 54							
	7	Thread 39								Thread 55							
	8	Thread 40								Thread 56							
	9	Thread 41								Thread 57							
	10	Thread 42								Thread 58							
	11	Thread 43								Thread 59							
	12	Thread 44								Thread 60							
	13	Thread 45								Thread 61							
	14	Thread 46								Thread 62							
	15	Thread 47								Thread 63							

Figure 1 Matrix A Row Major A[16][32] or B Column Major B[32][16] (16b)

```
- register=1 -> (0, 1)
  register=2 -> (0, 2)
  register=4 -> (0, 4)
- lane=1 -> (1, 0)
  lane=2 -> (2, 0)
  lane=4 -> (4, 0)
  lane=8 -> (8, 0)
  lane=16 -> (0, 8)
  lane=32 -> (0, 16)
[dim0 (size 16), dim1 (size 32)]
```

Linear Layout for ds_read_tr16_b64

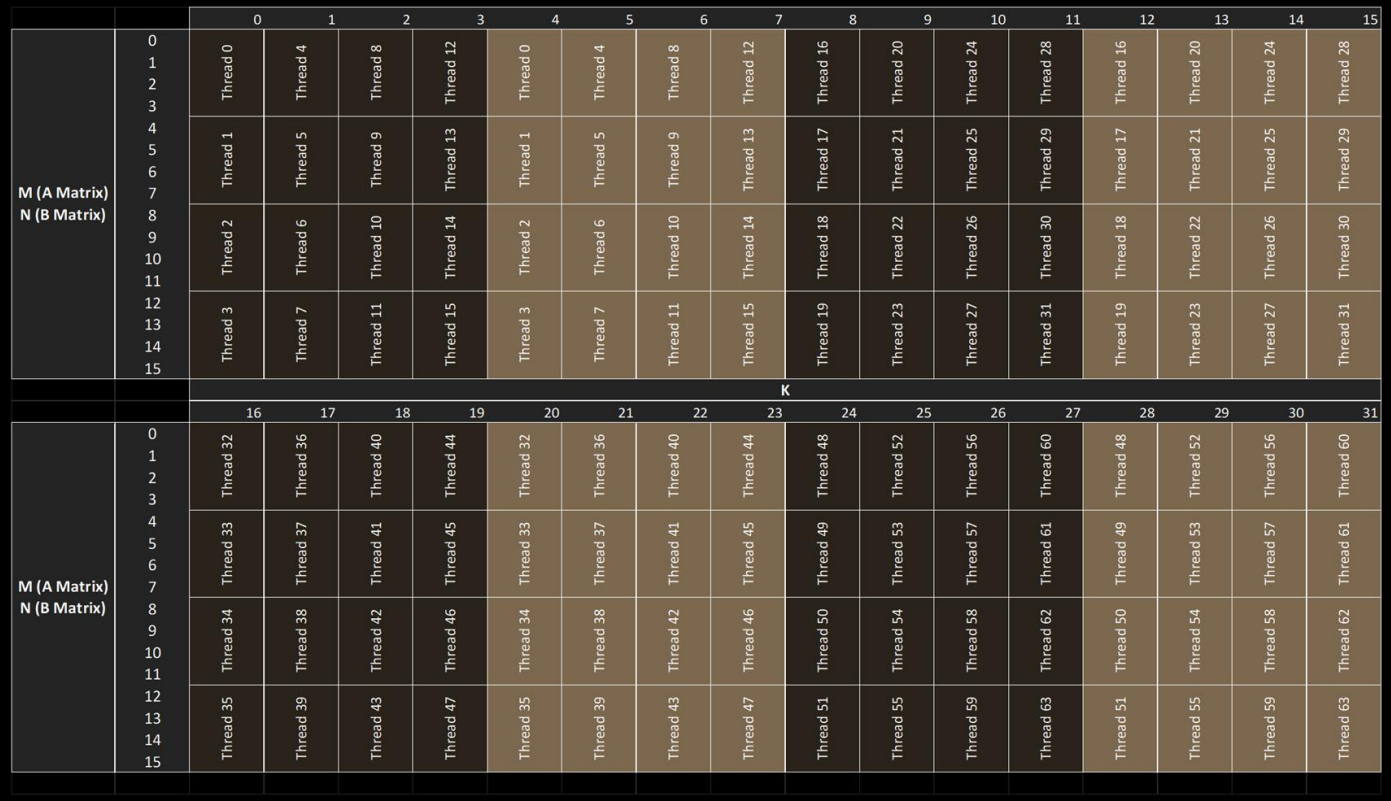


Figure 3 Matrix A Column Major [16][32] or Matrix B Row Major [32][16] (16b)

```
- register=1 -> (1, 0)
  register=2 -> (2, 0)
  register=4 -> (0, 4)
- lane=1 -> (4, 0)
  lane=2 -> (8, 0)
  lane=4 -> (0, 1)
  lane=8 -> (0, 2)
  lane=16 -> (0, 8)
  lane=32 -> (0, 16)
[dim0 (size 16), dim1 (size 32)]
```

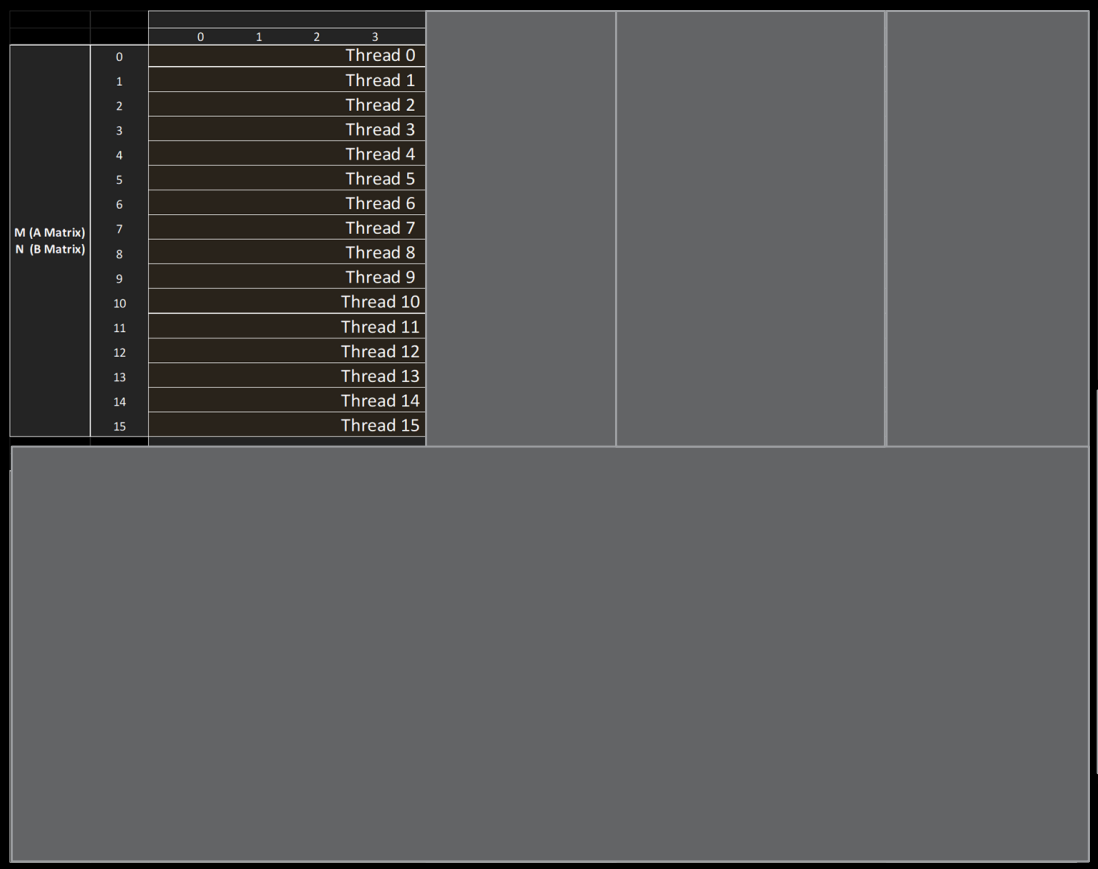
Using this layout will result in the instruction returning an MFMA layout

How do we lower local_load?

1. Take destination layout (DotOp with MFMA) and shared layout (SwizzleShared), check a lot of preconditions
2. Based on those layouts, generate from scratch a linear layout for the whole load. Quite an involved process
3. Create the address map: ldsTransLayout.invertAndCompose(sharedLL)
4. Go through load lowering path with custom ds_read_tr inserter. This was using the legacy lowering (emitTransferBetweenRegistersAndShared).

The scope of my work was to generalize the process to work with Linear Layout.

Let’s analyze how the instruction works



- Each thread reads 4 elements and the data is shared in a group of 16 threads
- This is a much smaller restriction than requiring DotOp/MFMA for the whole tile
- Can only be expressed using Linear Layout
- Allows usage of ds_read_tr for cases of local_load + tt.trans

How do we create a LL for the load?

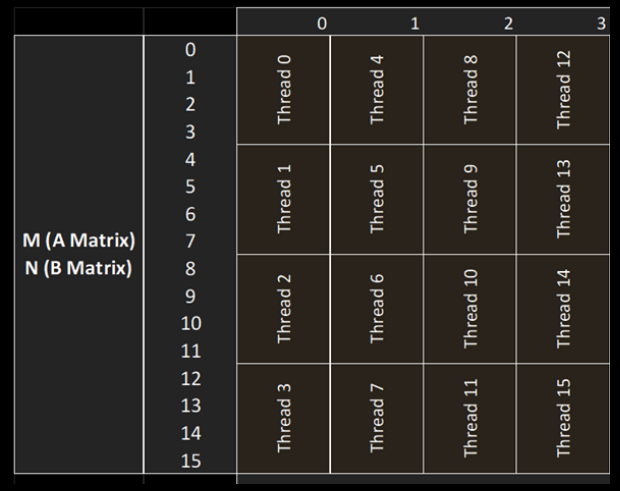
- We only need to bake in the inner tile
  - The first 2 register bases (4 contiguous register elements)
  - The first 4 lane bases (16 contiguous lanes)
- Old code generated the whole tile, can't do it anymore because the outer part of the tile might be different
- Idea: copy the destination LL but change the basis to what the instruction will be doing
- ds_read_tr shuffles the data in a way that can be described by a "left rotation" of bases between register and lane dimension for the inner tile
  - Register bases are always the first log2(N), with N = instBitWidth / elementBitWidth. So either 2 or 3 in gfx950
  - Lane bases is always the first log2(16) = 4

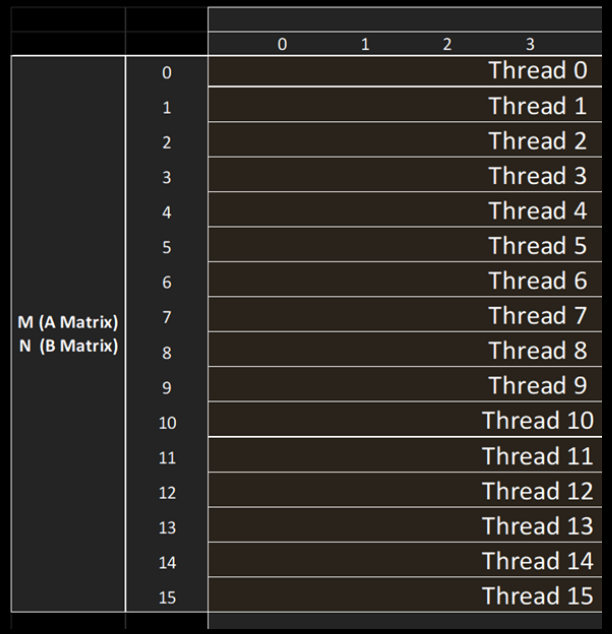
```
dstLL
 - register=1 -> (0, 1)
   register=2 -> (0, 2)
   register=4 -> (0, 4)
 - lane=1 -> (1, 0)
   lane=2 -> (2, 0)
   lane=4 -> (4, 0)
   lane=8 -> (8, 0)
   lane=16 -> (0, 8)
   lane=32 -> (0, 16)
[dim0 (size 16), dim1 (size 32)]
```

```
ldsTransLL
 - register=1 -> (1, 0)
   register=2 -> (2, 0)
   register=4 -> (0, 4)
 - lane=1 -> (4, 0)
   lane=2 -> (8, 0)
   lane=4 -> (0, 1)
   lane=8 -> (0, 2)
   lane=16 -> (0, 8)
   lane=32 -> (0, 16)
[dim0 (size 16), dim1 (size 32)]
```

Can we relax the contiguity restrictions?

- Does it really matter what logical elements are in the threads?
- Data is just data, the HW doesn't care about logical elements
  - No need to worry about it either!
  - As long as the data movement is correctly represented in the LL
- The only real requirement is that the composition of ldsTransLL and sharedLL can be vectorized to use the full instruction width
  - For ds_read_tr16_b64 that is 4 elements (64 bits)
- With these changes, we can use ds_read_tr for B16 types of this layout






```
dstLL
 - register=1 -> (0, 64)
   register=2 -> (16, 0)
   register=4 -> (0, 1)
   register=8 -> (32, 0)
   register=16 -> (0, 2)
   register=32 -> (0, 4)
   register=64 -> (64, 0)
   register=128 -> (0, 8)
 - lane=1 -> (1, 0)
   lane=2 -> (2, 0)
   lane=4 -> (0, 16)
   lane=8 -> (4, 0)
   lane=16 -> (8, 0)
   lane=32 -> (0, 32)
```

```
ldsTransLL
 - register=1 -> (1, 0)
   register=2 -> (2, 0)
   register=4 -> (0, 1)
   register=8 -> (32, 0)
   register=16 -> (0, 2)
   register=32 -> (0, 4)
   register=64 -> (64, 0)
   register=128 -> (0, 8)
 - lane=1 -> (0, 16)
   lane=2 -> (4, 0)
   lane=4 -> (0, 64)
   lane=8 -> (16, 0)
   lane=16 -> (8, 0)
   lane=32 -> (0, 32)
```

How do we lower local_load now?

1. Take destination layout and shared layout as linear layout
2. Generate ldsTransLayout by rotating the destination's LL reg/lane bases
   - Need to ensure the vectorization factor is at least 64 bits (and is limited to that because of instruction width).
3. Create the address map: ldsTransLayout.invertAndCompose(sharedLL)
4. Go through load lowering path with custom ds_read_tr inserter.
   - This is now using the new lowering (lowerLdSt).

The process is now much simpler, flexible and easier to adapt for new HW

What about other data types?

- FP8 types in gfx950 work the same as the 16 bit data types, except vectorization needs more elements
  - Instruction is ds_read_tr8_b64, so each thread works on 8 elements
- FP6 hasn't been looked at yet (ds_read_tr6_b96)
- FP4 data types are a bit more tricky:
  - They are not Triton native types so they are represented as i8 types, this means the tensor will contain packed data. This can either be K-packed or M/N packed
  - When packed along K, we can use the same codepath as FP8 types (ds_read_tr8_b64)
  - When packed along M/N we have a special operation (local_load_packed_transposed) because the packing changes when doing a load.
    This changes the dimensions of the tensor such as: 16x64xi8 → 32x32xi8 (for A) and 64x16xi8 → 32x32xi8 (for B).

What about gfx1250?

- New instructions are similar to gfx950
  - ds_load_tr16_b128, ds_load_tr8_b64, ds_load_tr4_b64, ds_load_tr6_b96
- Support has already landed in Triton (PR 8461).
  - It uses the same rotation algorithm for the bases
  - Small adjustment needed because the instruction for B8 types behaves slightly differently (two lanes swapped)
    - This is because of how the scaled dot WMMA layout is
- FP4 support (packing changes) is not yet implemented as it’s low priority
- Global load instruction support isn’t there yet
  - global_load_tr16_b128, global_load_tr8_b64, global_load_tr4_b64, global_load_tr6_b96
  - Look similar enough to the LDS operations
  - Implemenation will probably reuse most of the code

Next steps (WIP)

- Currently our lowering is based on transforming the LL based on how the instruction works
- Lowering in NV is based on defining a layout independent tile/fullTile:
  - tile is the smallest chunk of data that an instruction can operate on freely
  - fullTile is how the entire instance of the instruction works when issued and how data is distributed
- The new lowering uses the LL to map directly how the instruction works
  - Then calculates addresses for each thread based on this
  - Retains the same flexibility as current approach

```
tile ds_read_tr8_b64 (gfx950)
 - register is a size 1 dimension
 - lane=1 -> (1)
   lane=2 -> (2)
   lane=4 -> (4)
[offset (size 8)]
```

```
fullTile ds_read_tr8_b64 (gfx950)
 - register=1 -> (0, 2)
   register=2 -> (0, 4)
   register=4 -> (0, 8)
 - lane=1 -> (1, 0)
   lane=2 -> (2, 0)
   lane=4 -> (4, 0)
   lane=8 -> (0, 1)
   lane=16 -> (0, 16)
   lane=32 -> (0, 32)
[offset (size 8), addr (size 64)]
```


AMD